



UNIVERSIDADE DA CORUÑA

Facultad de Informática

Grado en Inteligencia Artificial

Práctica 2: Sistemas Multiagentes

SMA

Rodrigo García Romero
Alberto Blanco Lamela

3 de mayo de 2026

Enlace del repositorio

<https://github.com/rodrigogarcia1/EntregaSMA.git>

Mundo Virtual

El mapa que hemos desarrollado para esta práctica combina zonas abiertas de exterior con un interior lleno de habitaciones y pasillos. El objetivo de este diseño es ponerle las cosas difíciles al jugador y forzar a los agentes a colaborar.

Este diseño tiene partes clave, pasillos y habitaciones además de zonas abiertas. Las zonas estrechas para que los agentes puedan acorralar al jugador y habitaciones conectadas por puertas. Las puertas son muy importantes porque, cuando están cerradas, cortan la visión por completo. Las zonas abiertas sirven para que los agentes o las cámaras vean al jugador desde lejos y den la voz de alarma rápido.

El entorno interactúa con los sensores de los agentes. La visión se ve anulada si hay un muro o una puerta por medio del agente y el ladrón. Por eso, la comunicación de la posición del jugador por mensajes es vital para no perderle la pista. El sonido hace que, si el jugador corre, los agentes cercanos se alerten aunque no lo vean. Esto hace que dejen de patrullar y vayan directos a investigar qué ha sido ese ruido. El tacto, si el jugador los roza entonces investigarán ese lugar.

Además de los guardias que patrullan, hemos metido cámaras de seguridad en puntos estratégicos. Ni se mueven ni oyen nada, solo ven. Su trabajo es vigilar las zonas importantes (donde están las reliquias o los pasillos principales). En cuanto ven al jugador, mandan un mensaje con su posición.

Arquitectura individual

La arquitectura usada fue una arquitectura subsumida, de la misma manera que nuestra anterior iteración de la práctica.

El sistema utiliza una arquitectura que combina el modelo reactivo de **subsunción** con coordinación colectiva mediante comunicación.

Tanto agentes como agentes_camara comparten el mismo tipo de procesador de mensajes y comportamientos, **CerebroSubsumido**, aunque las cámaras actúan solo como sensores que informan sin ejecutar acciones. Los comportamientos se organizan jerárquicamente por prioridad:

Chase >Registrar >Cerco >Flanqueo >Ir a Reliquia >... >Patrulla

El funcionamiento se basa en sensores (**Vision**, **Hearing**, **Touch**) que notifican al cerebro para que se gestione una cola de comportamientos según **cumplePrecondiciones**. Los niveles superiores subsumen a los inferiores.

La comunicación se integra actualizando la **BaseConocimiento** local. Tras detectar al jugador, un iniciador lanza una subasta (**CFP**) y asigna roles (**ACCEPT_PROPOSAL**) basados en el coste por distancia informado por cada agente.

De esta manera los agentes se pueden coordinar con FIPA para la asignación de misiones (**ChasePlayer**, **FlanqueoPlayer**, etc.) que los comportamientos de subsunción ejecutarán. Una vez cumplida la misión u agotado su tiempo, el agente limpia su estado interno y retoma automáticamente tareas de menor prioridad como la patrulla o defensa de reliquias.

Comunicación entre agentes

La comunicación entre agentes se implementa mediante una clase **Message**, inspirada en el estándar FIPA-ACL. Cada mensaje contiene:

- **Performativa:** Define la intención del mensaje (**INFORM**, **CFP**, **PROPOSE**, **ACCEPT_PROPOSAL**, **REFUSE**).
- **Tipo (Message _Types):** Especifica el contenido semántico (por ejemplo, **PlayerSeen**, **ChasePlayer**, **FlanqueoPlayer**, **CercoPlayer**, **Reliquia-Robada**).
- **Sender/Receiver:** Identificación de los agentes implicados.
- **ConvID:** Identificador de conversación para agrupar mensajes dentro de un mismo protocolo (subasta).
- **Datos:** Información asociada al mensaje, como la posición (**Vector3**) del jugador o la distancia de cada agente.

El sistema sigue un flujo de comunicación de la siguiente manera:

1. **Creación:** Los mensajes se generan como respuesta a eventos del entorno, principalmente desde los sensores (por ejemplo, detección del jugador mediante **Vision** o robo de una reliquia).
2. **Envío:** Los mensajes se envían entre los agentes registrados en **baseConocimiento.agentes**, evitando comunicaciones innecesarias fuera del sistema multiagente (intervenciones de cámaras como patrullas por ejemplo).

3. **Recepción:** Cada agente dispone de una cola de mensajes entrantes que se procesa periódicamente con `ProcessMessages` del `CerebroSubsumido`.
4. **Procesamiento:** Los mensajes se interpretan mediante estructuras `switch-case`, actualizando la `BaseConocimiento` del agente (por ejemplo, asignando una misión o actualizando la posición objetivo).

Se han implementado dos protocolos principales:

Uno basado en la performative **INFORM**, permite compartir eventos relevantes:

- **PlayerSeen:** Un agente o cámara detecta al jugador y comunica su posición al resto.
- **ReliquiaRobada:** Se notifica la desaparición de una reliquia para actualizar el conocimiento global.

La otra implementándose la subasta de la siguiente manera para coordinar la persecución del jugador:

1. Un único agente lanza un **CFP** tras ver al jugador.
2. Los agentes responden con **PROPOSE**, indicando su coste (distancia al objetivo).
3. El iniciador evalúa las propuestas y asigna roles mediante **ACCEPT_PROPOSAL**:
 - Mejor coste → **Chase**
 - Segundo → **Flanqueo**
 - Resto → **Cerco**

Las cámaras pueden iniciar la comunicación mediante **INFORM**, pero no participan en la subasta.

Formato del contenido de los lenguajes

El contenido de los mensajes se representa mediante una estructura de datos en lenguaje `C#`, evitando el uso de texto libre y evitando la necesidad de parseo. Cada mensaje encapsula directamente los datos relevantes del entorno a través de los siguientes campos:

- **Performative:** Intención del mensaje. Los valores posibles son `Inform`, `CFP`, `Propose`, `AcceptProposal` y `Refuse`.
- **Tipo semántico** (`Message_Types`): Campo del mensaje que indica el dominio: `PlayerSeen`, `PlayerHeard`, `PlayerLost`, `ReliquiaRobada`, `ChasePlayer`, `FlanqueoPlayer` y `CercoPlayer`.
- **Sender / Receiver** (`GameObject`): Identifican al emisor y, opcionalmente, al receptor.
- **Position** (`Vector3`): Coordenadas del jugador o de la reliquia robada.
- **Valor de propuesta** (`proposalValue`): Coste incluido en los mensajes `Propose` para comparar ofertas durante la subasta.
- **Referencia a reliquia** (`reliquia`): Campo `GameObject` específico para identificar la reliquia concreta afectada en mensajes `ReliquiaRobada`.
- **Identificador de conversación** (`ConvID`): Número de identificación de la conversación.
- **Marca de tiempo** (`timestamp`): Registra el momento de la creación del mensaje mediante.

El procesamiento de los mensajes se maneja desde `ProcessMessages` del `CerebroSubsumido`, siguiendo el siguiente modelo:

1. **Filtrado:** Mediante una estructura `switch-case` sobre la performativa y el tipo de mensaje, el agente determina qué lógica aplicar.
2. **Extracción:** Los datos relevantes se leen directamente de los campos del mensaje sin transformaciones adicionales. Por ejemplo, `msg.position` proporciona la posición del jugador y `msg.proposalValue` el coste de una propuesta.
3. **Actualización del estado:** La información extraída se almacena en la `BaseConocimiento` local. Un `AcceptProposal` actualiza `misión` y `MissionTarget`; un `ReliquiaRobada` elimina la reliquia referenciada en `msg.reliquia` de la lista `relicList`.
4. **Activación por subsunción:** Los comportamientos consultan la `BaseConocimiento` en sus `cumplePrecondiciones` y reaccionan al nuevo estado en el siguiente ciclo de ejecución.

Así se garantiza que la comunicación sea fluida, los mensajes actualizan la base de conocimiento de cada agente y los comportamientos reaccionan acorde a ello, manteniéndose independientes entre sí pero cooperando al seguir las estrategias para capturar al jugador.

Estrategia Multiagente

Los agentes se comunican entre sí mediante mensajes en una comunicación tipo P2P, donde el agente que inició la comunicación se encarga de manejar las respuestas de todos los demás agentes en caso de que la comunicación inicial fuera múltiple.

Los agentes hacen planes cooperativos principalmente mediante el uso de subastas de comportamientos. En estas se reparten uno o varios roles de un mismo tipo o tipos distintos para poder realizar combinaciones de comportamientos que asemejen inteligentes. Las subastas se gestionan mediante el protocolo ContractNet, siendo el iniciador de estas el encargado de repartir los roles según las respuestas de los participantes.

Los tipos de roles que se pueden asignar durante un CFP cuando se encuentra a un jugador son Flanqueador, Chaser y Cercador.

Los Flanqueadores intentan atrapar al jugador yendo a habitaciones contiguas mientras el Chaser lo persigue directamente. Los Cercadores intentan delimitar el rango de acción del jugador poniéndose a modo de muralla por la zona en la que se avistó al ladrón. De esta forma se intenta maximizar las probabilidades de que se atrape al ladrón minimizando sus opciones de huida.